深圳大学传播学院

网络与新媒体 专业本科毕业设计

个人研究报告

题　　目：AI 编程范式驱动的数字传播产品开发实践

——以传播艺术项目《显·影》为例

姓　　名：曹思越

学　　号：2022080006

小组项目：互动、在场与传播：传播艺术项目《显·影》的设计与实现

指导教师：李梓音

年　　月　　日

深圳大学本科毕业论文个人研究报告

诚 信 声 明

本人郑重声明：所呈交的毕业设计个人研究报告，题目《AI 编程范式驱动的数字传播产品开发实践——以传播艺术项目《显·影》为例》是本人在指导教师的指导下，结合项目小组设计工作实践，独立进行研究工作所取得的成果。本报告与项目小组集体完成之设计项目相配合，报告所引述之相关资料及文献，均在报告中以明确方式说明。本人完全意识到本声明的法律结果。

作者签名：

日期：　　年　　月　　日

目录

1 引言 4
2 文献综述 5
2.1 AI 编程范式的兴起与效率实证 5
2.2 从提示词到上下文：AI 代码生成质量的影响因素 5
2.3 非技术开发者与全链路实践的研究空白 5
3 研究方法 5
3.1 研究设计 5
3.2 数据收集 5
3.3 分析框架 6
3.4 数据编码与分析 6
4 开发实践过程 6
4.1 系统设计：产品思维驱动的技术决策 6
4.2 上下文工程：让 AI 理解项目的策略体系 7
4.3 平台适配：真实部署环境中的工程挑战 7
4.4 数据 Pipeline：从原始语音到互动可视化的全链路设计 8
4.5 视觉表达：从数据到艺术的转译实践 8
4.6 迭代修正：应对 AI 生成偏差的策略 9
5 发现与讨论 10
5.1 效率的不均匀分布及其成因 10
5.2 "物质脱嵌"的层次性 10
5.3 数字传播视角：当技术门槛被降低后 10
6 结论 10

AI 编程范式驱动的数字传播产品开发实践

——以传播艺术项目“显·影”App 为个案

曹思越
（深圳大学传播学院 网络与新媒体专业 2022080006）
指导教师：李梓音

【摘要】在生成式人工智能技术快速迭代的背景下，以自然语言交互为核心的"氛围编程"（Vibe Coding）范式正深刻改变软件开发的门槛与流程。本研究以传播艺术项目"显·影"App 的完整设计与开发过程为个案，采用行动研究与个案实践相结合的方法，系统记录并分析了一名网络与新媒体专业本科生如何借助 AI 代码编辑器 Cursor，通过人机协作完成从设计稿到可运行 iOS 应用的全链路产品落地。研究基于 28 轮人机对话记录和 12 份技术计划文档，从系统设计、上下文工程、平台适配、数据 Pipeline 构建和视觉表达五个维度提炼了 Vibe Coding 实践中的人机协作策略。研究发现，AI 编程范式在界面搭建、接口对接等模式化任务中可显著提升开发效率，但在系统架构决策、跨平台兼容性调试等需要全局判断的环节仍高度依赖开发者的介入。本文进一步提出，当技术实现的门槛被 AI 降低后，数字传播产品的核心竞争力将加速向传播创意、用户洞察和审美判断迁移，这对网络与新媒体专业的人才培养具有重要启示。

【关键词】Vibe Coding；生成式 AI；人机协作开发；数字传播产品；Cursor

1 引言

2024 年以来，以 ChatGPT、Claude、Gemini 为代表的大语言模型在代码生成领域的能力出现跨越式提升。特斯拉前 AI 总监 Andrej Karpathy 提出"Vibe Coding"概念后，一种全新的编程范式引发学界和业界广泛关注：开发者通过自然语言向 AI 传达意图，由 AI 生成具体实现方案[1]。这一范式使软件开发的门槛发生结构性变化，过去需要数年计算机科学训练才能完成的工程任务，如今有可能由具备基本逻辑思维的非技术背景从业者通过人机协作来实现。

然而，Vibe Coding 的实际效能边界尚不明晰。既有研究主要集中于效率量化和提示词策略优化[4][5]，非计算机专业背景的开发者如何利用 AI 编程范式独立完成一个从概念到上线的完整产品，相关深度个案仍然匮乏[6]。数字传播领域的产品开发更为特殊，因为它同时涉及技术实现、交互设计、用户体验与传播逻辑，对 AI 编程范式提出了更复合的要求。一个更根本的问题是：当 AI 将技术门槛大幅降低后，开发者的核心价值究竟体现在何处？

"显·影"是深圳大学传播学院的一项毕业设计项目，其核心命题是：在群体互动场景中，大量互动细节（谁在对谁说话、发言间的打断与沉默、语调变化与氛围转折等）难以被传统记录方式捕捉和复盘。项目的愿景是将这些不显著的互动信息提取为结构化数据，并通过三维可视化的方式加以呈现，帮助用户直观地理解互动关系、互动强度与情绪氛围。笔者作为该项目的产品设计与技术开发负责人，需要将这一抽象愿景转化为一个可运行的 iOS 应用产品，覆盖从语音录制、数据处理到可视化生成的完整链路。作为网络与新媒体专业的本科生，笔者全程依托 AI 代码编辑器 Cursor，采用 Vibe Coding 的方式完成了 App 从设计到部署的全部技术工作。这一经历使笔者得以从实践者视角观察 AI 编程范式在真实项目中的能力边界与协作模式。

本研究的目的在于：以"显·影"App 的开发实践为个案，系统记录 AI 编程范式在数字传播产品开发各环节中的实际作用与局限，提炼人机协作中开发者的策略性思维模式，为非技术背景开发者的 AI 协作路径和数字传播领域的产品教学提供经验参照。

2 文献综述

2.1 AI 编程范式的兴起与效率实证

围绕 AI 辅助编程的实际效率，学界已形成初步共识：该技术对模式化任务的加速效果显著，但对需要全局判断的任务帮助有限。微软研究院的 Peng 等人通过大规模控制实验发现，使用 GitHub Copilot 的开发者完成编码任务的速度比未使用者快 55.8%[7]；GitHub 数据科学团队的 Ziegler 等人对 93 万余名用户的追踪分析进一步显示，用户接受了近 30%的 AI 代码建议，且普遍报告生产力提升，但研究同时指出，AI 的核心价值在于提供"可迭代的起点"，开发者仍需在此基础上进行修正和完善[9]。

在编程范式层面，微软研究院的 Sarkar 和 Drosos 通过对 8 小时编程录像的框架分析，首次系统描述了 Vibe Coding 的工作模式：开发者在"提示-评估-修正"三个阶段间反复切换，形成所谓"物质脱嵌"（material disengagement），即通过 AI 中介编排代码的生产，同时保持选择性的战略监督[1]。独立研究者 Hemdev 的混合方法案例研究呼应了这一发现，指出开发者需要在"委托"与"控制"之间持续寻找平衡，过度依赖 AI 会导致代码库的架构一致性下降[6]。

2.2 从提示词到上下文：AI 代码生成质量的影响因素

影响 AI 代码生成质量的因素，既有研究从两个方向展开了探讨。韩国庆熙大学的 Bae 等人在计算机安全领域的实验表明，采用分步推理（Chain-of-Thought）或角色设定（Persona-based）的结构化提示策略，能将 GPT-4o 和 Claude-3.5 Sonnet 的代码漏洞检测准确率提升 15 至 25 个百分点[5]。国内学者许翔和蔡宁楠则从工程应用角度指出，AI 输出中的"幻觉"问题，即生成看似合理却无法运行的代码，构成了实际开发中的主要风险，开发者必须具备足够的判断力进行甄别[3]。

在设计到代码的转换环节，前端工具平台 Builder.io 的调研显示，79%的开发者报告将 Figma 设计转化为代码需要超过一天的工作量，AI 转换工具可将这一过程缩短 50%至 80%[10]。但多项研究共同指出了一个规律：AI 生成内容的质量高度依赖输入的上下文质量，精确的技术规格和参考示例比任何修辞技巧都更有效[3][5][10]。

2.3 非技术开发者与全链路实践的研究空白

综合既有文献可以发现，当前研究以专业开发者为主要对象，在受控实验环境中进行效率量化。国内学者余可然对 AI 代码编辑器的方法探索[4]和郭昱辰、赵欣对 AI 驱动原型设计的研究[2]虽然开始触及设计领域，但均停留在单一环节的分析上。至于非计算机背景开发者如何利用 AI 编程范式，在真实项目中完成从概念设计到跨平台部署的全链路产品落地，深度个案几乎空白。此外，迈阿密大学信息可视化学者 Cairo 提出的"功能优先于装饰"原则[12]和美国马里兰艺术学院设计教育学者 Lupton 关于 AI 时代人类角色从"执行者"转向"策展者"的判断[13]，均为本研究提供了传播学维度的分析框架，但尚未在 AI 编程的语境中被实证检验。本研究试图从实践层面填补这些空白。

3 研究方法

3.1 研究设计

本研究采用行动研究法与个案实践法相结合的研究设计。行动研究法强调研究者以实践参与者的身份嵌入过程，在"计划—行动—观察—反思"的循环中推进认知[11]。笔者既是"显·影"App 的开发者，也是开发过程的记录者和分析者，这种双重身份使研究能够深入捕捉 AI 编程实践中的细粒度决策过程。

3.2 数据收集

本研究的数据来源包括三个维度：

第一，人机对话记录。笔者使用 Cursor AI 编辑器进行了 28 轮完整的人机协作对话，总计超过 3000 轮次交互。这些对话通过 SpecStory 扩展自动保存为结构化文档，每轮对话时间跨度从 30 分钟到 6 小时不等，涵盖需求描述、方案讨论、代码生成、错误调试和方案迭代等完整协作周期。

第二，技术计划文档。开发过程中积累了 12 份技术方案文档，涵盖语音识别接入、数据标注平台集成、三维可视化方案、跨平台打包等核心模块。这些文档在开发过程中由 AI 辅助生成并经笔者审核确认，具有较高的过程真实性。

第三，项目工程文件。包括完整源代码（76 个依赖包、9 个页面组件）、架构文档和产品愿景文档，为还原技术细节提供了客观支撑。

3.3 分析框架

本研究从人机协作策略的角度，将开发实践归纳为五个维度：系统设计决策、上下文工程策略、平台适配实践、数据 Pipeline 构建和视觉表达转译。每个维度重点分析开发者的策略性思维、AI 的能力边界以及两者之间的交互模式。

3.4 数据编码与分析

为量化人机协作的模式特征，笔者对 28 轮对话记录进行了内容编码，将每次交互按意图分为六类。编码结果如表 1 所示。

表 1 对话交互类型统计

交互类型	占比	平均迭代轮次	AI 首轮可用率
UI 界面调整	32%	1.5 轮	78%
后端功能实现	24%	3.2 轮	45%
错误调试	19%	4.8 轮	22%
架构决策	11%	5.5 轮	15%
文档查询	8%	1.2 轮	85%
部署配置	6%	3.0 轮	35%

（注：数据基于对 28 轮对话记录的回溯编码，"首轮可用率"指 AI 第一次生成即可直接使用的比例）

该统计揭示了一个关键规律：AI 的首轮可用率与任务的模式化程度高度正相关。UI 调整和文档查询等高度模式化的任务超过 78%，而架构决策和错误调试等需要全局理解的任务不足 25%。

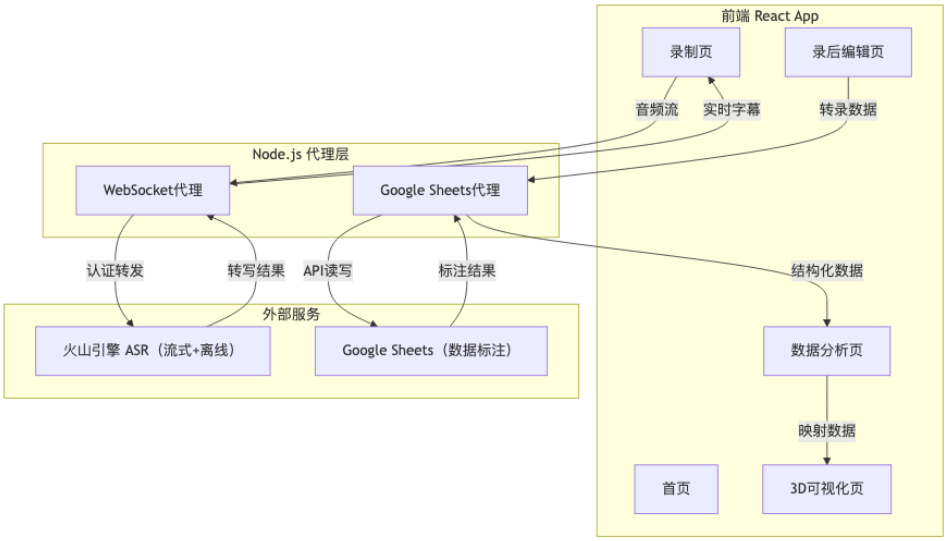
4 开发实践过程

"显·影"App 的产品目标是录制群体对话，将语音转化为结构化的互动数据，再生成三维可视化作品，帮助用户以直观方式理解群体互动中的关系结构与情感流动。用户的核心流程分为四步：录制对话、校对转写文本、AI 辅助数据标注、生成三维可视化。围绕这条产品主线，笔者作为产品设计者和唯一的技术搭建者，需要完成五个技术模块的开发：前端界面（9 个页面组成的移动端 App）、语音识别服务接入、数据标注管线、三维可视化引擎、以及 iOS 跨平台打包。以下从人机协作策略的维度，分别讨论这些模块在 Vibe Coding 实践中的关键经验。

4.1 系统设计：产品思维驱动的技术决策

在 Vibe Coding 范式下，AI 已能高效完成大部分代码编写工作，开发者的核心价值集中体现在技术方案的判断与取舍上。"显·影"App 的开发过程中，笔者做出了三项关键的系统级决策，这些决策均依赖于对项目约束条件的全局理解，无法由 AI 独立完成。

图 1 系统架构图



4.2 上下文工程：让 AI 理解项目的策略体系

系统设计阶段确定了"做什么"和"用什么做"的问题，接下来的挑战是：如何让 AI 在具体编码中正确理解并执行这些技术决策？

在回溯 28 轮开发对话后，笔者发现一个核心规律：AI 生成代码的质量取决于开发者向 AI 供给的上下文质量。学术研究关注 Chain-of-Thought 等结构化提示技术[5]，而真实工程实践中最有效的策略往往是直接向 AI 提供正确的技术文档、协议规格和代码示例。笔者将这一实践归纳为"上下文工程"（Context Engineering），并提炼出四层策略。

表 2 四种上下文策略概览

策略	适用场景	核心逻辑
文档投喂	外部接口/协议实现	精确规格 > 自然语言描述
参考驱动示例	设计意图/视觉风格	将低质量素材转化为 AI 可学习的示例
架构约束前置	长对话一致性维护	每轮重申技术栈和架构规范
沙箱隔离开发	高不确定性模块	隔离试错，保护主项目稳定性

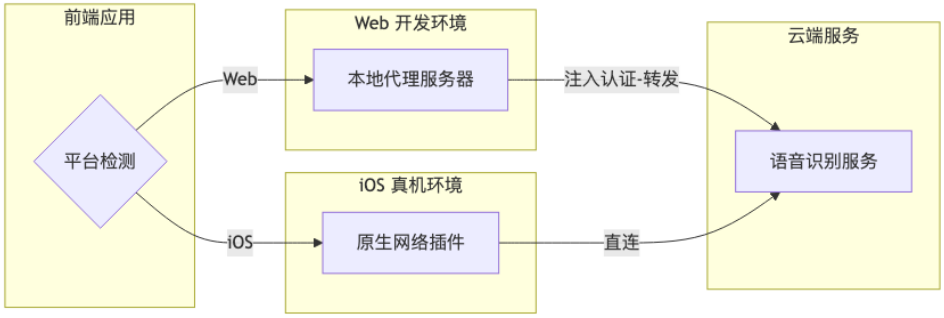
4.3 平台适配：真实部署环境中的工程挑战

上下文工程策略解决了 AI 代码生成的质量问题，但生成的代码从开发环境迈向真实设备时，还面临着 AI 难以感知的平台差异与安全限制。

AI 擅长生成在标准开发环境中运行的代码，但真实部署面临的平台差异和安全限制往往超出 AI 的感知范围。在"显·影"的开发中，平台适配消耗了大量调试精力，也是 AI 首轮可用率最低的领域。

最典型的案例涉及浏览器安全限制。语音识别服务要求在网络连接建立时携带认证信息，但浏览器出于安全考虑禁止了这一操作。AI 最初生成的方案在桌面开发环境中可行，但在浏览器中被静默忽略。笔者向 AI 描述这一差异后，AI 提出了中间代理方案，由服务端代为完成认证。然而，这个方案在移动端又引发第二层问题：打包后的 iOS App 中没有服务端运行环境。经过三轮方案迭代，最终确定了按平台分流的策略。

图 2 平台适配分流架构



这一"问题-方案-新问题-再迭代"的过程历时两天。AI 在每一轮中都能给出技术上可行的局部方案，但将局部方案整合为跨平台一致的架构需要开发者持有全局视角。跨平台部署中还涉及大量操作性问题：屏幕安全区域适配、应用签名配置、开发者证书过期处理等。这类问题的共同特征是它们属于真实部署中的边缘情况，AI 在开发者准确描述现象后能给出有效指引。

4.4 数据 Pipeline：从原始语音到互动可视化的全链路设计

平台适配打通了 App 在不同环境下的基础运行能力，下一步是构建产品的核心功能，即将原始语音数据加工为可供可视化使用的结构化信息。

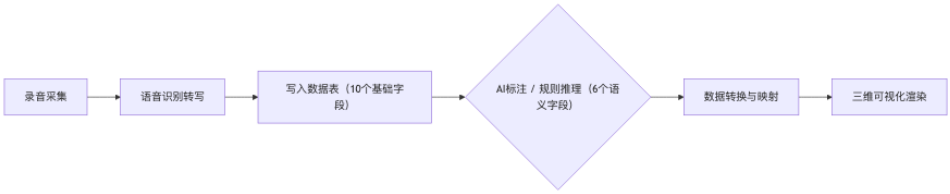
"显·影"的产品核心是一条将群体对话录音转化为互动可视化作品的数据 Pipeline。这条管线的设计逻辑反映了数字传播产品的根本命题：技术只是手段，"将什么信息以什么方式呈现给用户"才是核心价值。

笔者设计了一套包含 16 个字段的数据结构：10 个基础特征字段（发言内容、说话人、时间戳、时长、性别、情绪、语速、音量等）由语音识别引擎直接输出或通过简单计算得到；6 个语义标注字段（对谁发言、对话情绪、是否被打断、互动类型、话题标签、对话转折点）则需要通过 AI 或规则推理进行填充。

在标注方案上，笔者采用了"双层容错"架构：第一层利用 Google Sheets 的 AI 公式进行语义分析自动标注，第二层在代理服务器中实现了基于规则的推理引擎作为备选。之所以设计双层架构，是因为测试中发现 AI 公式存在工程化限制，通过接口写入的公式无法自动触发计算，需要用户在浏览器中手动打开才会激活。规则推理层确保了无论 AI 服务是否可用，数据 Pipeline 都能持续运转。这种"主方案+降级方案"的容错设计，恰恰是 AI 难以自主提出的，因为 AI 倾向于给出单一最优方案，较少考虑方案失效时的备选路径。

从原始字段到可视化元素的映射设计，需要传播学维度的思考。信息可视化理论强调视觉编码应利用人类感知直觉[12]：情绪基调映射为连线颜色（绿色积极、红色消极、蓝色中性），借用了色彩心理学中的普遍联想；音量映射为连线粗细，利用了视觉权重的隐喻；对话转折点触发涟漪特效，通过突然的视觉变化传达氛围的急剧转换。这些映射决策本质上是传播学中"信息编码"问题的可视化实践，它们由笔者基于产品理解主导设计，AI 仅负责技术实现。

图 3 端到端数据流



4.5 视觉表达：从数据到艺术的转译实践

数据 Pipeline 产出了包含 16 个字段的结构化对话数据，最后一步是将这些数据转化为用户可感知的三维可视化作品，这也是"显·影"作为传播艺术项目的最终表达。

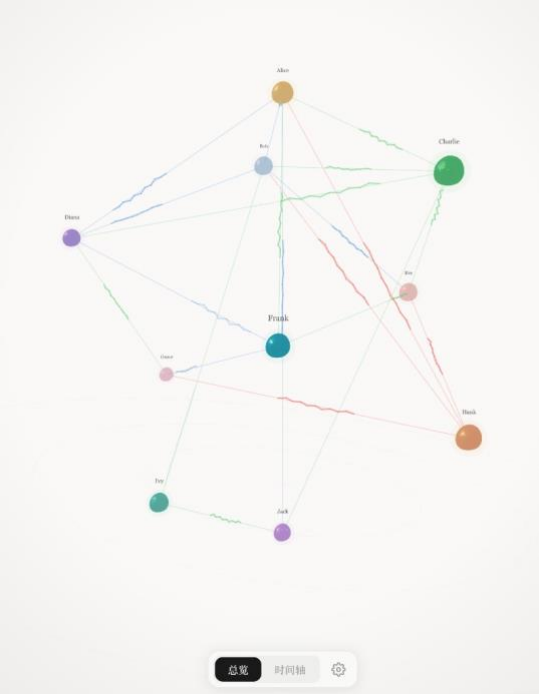
可视化模块的目标是将结构化的对话数据转化为三维交互图谱，让用户以直观的方式观察群体互动的结构、强度和情感色彩。每位参与者呈现为一个节点，发言连接呈现为带有方向和色彩的光束，粒子沿连线流动形成动态的"对话风暴"视觉效果。

这一模块的开发充分体现了沙箱隔离策略的价值。笔者首先在独立的测试目录中进行视觉风格探索：向 AI 提供参考网站的截图，并用意象性词汇描述期望效果。AI 展现了将抽象视觉意象转化为具体技术参数的能力，笔者无需指定渲染引擎的参数细节，只需描述"深空网络""柔和光晕"等意象，AI 即可生成对应的视觉方案。经过多轮探索和筛选，项目最终确定了两套视觉风格：一套偏向科技感的锐利风格，一套偏向自然感的柔和风格，用户可根据数据特征和表达意图进行切换。确认效果满意后，笔者才将代码移植到主项目中。

此外，笔者设计了三种交互模式，它们的逻辑来自对产品使用场景的理解：焦点模式让研究者聚焦特定参与者的互动网络，时间轴过滤支持对比不同时段的对话结构变化，关系过滤允许观察选定人员之间的互动流。这些交互设计由笔者基于用户需求主导，具体的渲染实现则高效地交由 AI 完成。

图 4a 沙箱隔离环境中的视觉风格探索

图 4b 最终选定的 3D 互动可视化效果



4.6 迭代修正：应对 AI 生成偏差的策略

在开发实践中，笔者逐渐建立了一套应对 AI 生成偏差的修正策略。

AI 的"审美偏差"是一个突出问题。在要求 AI 优化界面时，AI 倾向于添加大量渐变、阴影和装饰性动画，形成看似精美却缺乏克制的视觉风格。字体体系的重构是一个典型案例：AI 的初始方案在所有页面使用衬线字体，视觉上有设计感，但在移动端小字号下中文可读性很差。笔者参考了 Claude 官网等优秀产品的字体策略，确立了"衬线体仅用于品牌标题，正文使用无衬线体"的分层体系，经历四轮迭代才完全统一。启示是：开发者需要建立明确的设计标准并反复向 AI 重申，否则 AI 的默认输出会逐渐侵蚀产品的设计一致性。

AI 生成不存在的接口参数同样造成了显著的调试成本。在语音识别接入过程中，AI 生成了一个看似合理但实际不存在的功能参数，语法完全正确，只有在实际调用被服务端拒绝后才能发现。笔者的应对策略是：对于所有涉及外部服务的代码，一律对照官方文档逐一验证 AI 生成的接口调用方式，将"信任但验证"作为基本协作原则。

5 发现与讨论

5.1 效率的不均匀分布及其成因

基于表 1 的对话编码数据，AI 编程的效率提升在不同任务类型之间呈现显著的不均匀分布。UI 搭建类任务的 AI 首轮可用率高达 78%，估计节省了 70%至 80%的开发时间，与微软研究院关于 GitHub Copilot 效率的实证结论一致[7]。而架构决策类任务的首轮可用率仅为 15%，平均需要 5.5 轮迭代。

这一分布的成因可以从 AI 的能力结构来理解。AI 擅长模式匹配和知识检索，当任务可以归结为"在既有模式库中找到最匹配的实现"时表现优异；但当任务需要在多个约束条件之间权衡取舍时（如技术栈选型、数据标注平台迁移），AI 缺乏对项目全局状态的持续感知，难以给出最优解。

5.2 "物质脱嵌"的层次性

微软研究院的 Sarkar 和 Drosos 提出的"物质脱嵌"概念[1]在本研究的实践中呈现出值得细分的层次性。

在代码层面，脱嵌是充分且有效的。笔者在整个开发过程中从未使用浏览器调试工具进行断点调试，也很少逐行阅读 AI 生成的代码。"描述问题-AI 生成修复-运行验证"的循环构成了日常工作的基本节奏。

但在架构层面，脱嵌会带来风险。涉及系统级决策时，例如选择什么技术栈、设计怎样的容错架构、如何在多个平台间分流，开发者必须跳出 AI 生成的具体代码，从全局约束条件出发进行判断。AI 缺乏对团队能力、时间成本、平台限制和产品优先级的持续感知，这些决策无法被外包。换言之，Vibe Coding 中开发者的角色更接近"技术产品经理"，核心能力是做正确的决策。

本研究的实践还表明，非技术背景开发者在 Vibe Coding 中需要三项关键能力：**问题拆解能力**，将模糊需求拆解为 AI 可理解的原子化指令；**错误诊断能力**，从运行错误中提取关键线索并补充上下文反馈给 AI；**架构判断力**，在 AI 给出多个方案时评估哪个更适合当前项目约束。这三项能力均来源于对产品全局的理解。

5.3 数字传播视角：当技术门槛被降低后

本研究提示了一个对网络与新媒体专业具有重要意义的趋势：当 AI 将技术实现的门槛大幅降低后，数字传播产品的核心竞争力正加速从"技术实现能力"向"传播设计能力"迁移。

在"显·影"的开发中，AI 高效地处理了界面搭建、接口对接、数据格式转换等技术性工作，但以下环节始终由笔者主导：产品愿景的定义、数据字段体系的设计、视觉映射的传播学考量、以及用户体验的审美把控。这些环节的共同特征是：它们需要的是对"用什么方式向谁传达什么信息"这一传播学核心命题的理解。

美国马里兰艺术学院的设计教育学者 Lupton 和 Phillips 指出，在 AI 辅助创意工作中，人类的核心角色正从"执行者"转向"策展者"，即决定什么值得被创造、以什么方式被呈现、为谁服务[13]。本研究的实践印证了这一判断：在 Vibe Coding 语境下，网络与新媒体专业学生的竞争力在于是否具备清晰的传播设计思维，包括能否提出值得解决的问题、设计有效的信息架构、并在 AI 生成的众多方案中做出符合传播逻辑的取舍。

6 结论

本研究通过"显·影"App 的完整开发实践，从系统设计、上下文工程、平台适配、数据 Pipeline 和视觉表达五个维度，系统分析了 AI 编程范式在数字传播产品开发中的可行性与边界。

研究发现，Vibe Coding 范式确实显著降低了软件开发的准入门槛，使非技术背景的开发者能够完成从设计到部署的全链路产品落地。但 AI 编程并非全自动化过程，开发者的核心价值集中在技术方案的判断与取舍上。本研究提出的"上下文工程"策略体系（文档投喂、参考驱动示例、架构约束前置和沙箱隔离开发）为非技术背景开发者提供了可操作的实践框架。同时，研究揭示的效率

不均匀分布规律（模式化任务高效、决策性任务低效）为理解 AI 编程工具的能力边界提供了实证参考。

本研究存在以下局限：第一，作为单一个案，结论的可推广性受限；第二，笔者兼具研究者和开发者双重身份，可能存在主观性偏差；第三，对话编码中的首轮可用率基于回溯评估，缺乏实时记录的精确性。未来研究可扩大样本量对比不同 AI 工具的开发效率，引入客观代码质量指标进行纵向评估，以及探索 AI 编程范式在传播学、设计学等非计算机专业课程教学中的系统化应用。

参考文献

[1] Sarkar A, Drosos I. Vibe coding: programming through conversation with artificial intelligence[J]. arXiv preprint arXiv:2506.23253, 2025.

[2] 郭昱辰, 赵欣. 基于生成式AI驱动的产品概念原型设计[J]. 鞋类工艺与设计, 2025, 5(14): 33-35.

[3] 许翔, 蔡宁楠. 信息化产品智能化设计与开发中人工智能技术的应用研究[J]. 科技资讯, 2025, 23(20): 94-96.

[4] 余可然. 基于人工智能代码编辑器编写软件的方法探索[J]. 通讯世界, 2025, 32(06): 184-186.

[5] Bae J, Kwon S, Myeong S. Enhancing software code vulnerability detection using GPT-4o and Claude-3.5 Sonnet: A study on prompt engineering techniques[J]. Electronics, 2024, 13(13): 2657.

[6] Hemdev N. VIBECODING: A mixed-methods case study on conversational AI programming and application development[J]. IJSAT-International Journal on Science and Technology, 2025, 16(3).

[7] Peng S, Kalliamvakou E, Cihon P, et al. The impact of AI on developer productivity: Evidence from GitHub Copilot[J]. arXiv preprint arXiv:2302.06590, 2023.

[8] Cisco. GitHub Copilot productivity study: Real-world impact analysis[R]. Cisco DevOps Report, 2024.

[9] Ziegler A, Kalliamvakou E, Li X A, et al. Measuring GitHub Copilot's impact on productivity[J]. Communications of the ACM, 2024, 67(3): 54-63.

[10] Builder.io. Introducing Visual Copilot: A better Figma-to-code workflow[EB/OL]. https://builder.io/blog/figma-to-code-visual-copilot, 2024.

[11] 陈向明. 质的研究方法与社会科学研究[M]. 北京: 教育科学出版社, 2000.

[12] Cairo A. The functional art: An introduction to information graphics and visualization[M]. Berkeley: New Riders, 2013.

[13] Lupton E, Phillips J C. Graphic design: The new basics (revised and expanded)[M]. New York: Princeton Architectural Press, 2015.

致谢

感谢指导教师李梓音老师在选题设计与研究方向上的悉心指导。感谢"显·影"项目团队成员在产品定义、视觉设计和用户测试中的协作支持。感谢 Anthropic、OpenAI 和 Cursor 团队为本研究提供了核心的 AI 编程工具支撑。最后，感谢深圳大学传播学院为本次毕业设计提供的实践平台和学术资源。

【Abstract】Against the backdrop of rapidly evolving generative AI, the emerging "Vibe Coding" paradigm — centered on natural language interaction — is fundamentally reshaping the accessibility and workflow of software development. This study examines the full design-to-deployment process of "Xianying," a communication art project App, through action research combined with case practice methodology. It documents and analyzes how an undergraduate student majoring in Network and New Media leveraged the AI code editor Cursor to build a functional iOS application through human-AI collaboration. Based on 28 complete dialogue sessions and 12 technical planning documents, the study distills collaborative strategies across five dimensions: system design decisions, context engineering, platform adaptation, data pipeline construction, and visual expression. The findings reveal that AI programming significantly accelerates pattern-based tasks such as UI development and API integration, while remaining dependent on developer judgment for architectural decisions, cross-platform compatibility, and communication design logic. The study further proposes that as AI lowers the implementation barrier, the core competitiveness of digital communication products is shifting toward communication creativity, user insight, and aesthetic judgment.

【**Key words**】Vibe Coding; Generative AI; Human-AI Collaborative Development; Digital Communication Product; Cursor